

**LAPORAN PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
LINKED LIST**



DI SUSUN OLEH :

NAMA	: FARA MUTIA
NIM	: 2025573010034
PRODI	: TEKNIK INFORMATIKA
DOSEN PENGAJAR	: M. REZA ZULMAN, S.ST.,M.Sc

**JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
LHOKSEUMAWE**

2026

LEMBARAN PENGESAHAN

MATA KULIAH : Praktikum Struktur Data dan Algoritma
JUDUL PRAKTIKUM : Linked List
HARI PRAKTIKUM : Kamis
TANGGAL PENYERAHAN : 31 Mei 2026
NAMA : Fara Mutia
NIM : 2025573010034
PRODI/KELAS : Teknik Informatika/I D
NILAI :

Mahasiswa

Bukit Rata, 31 Mei 2026
Dosen Pembimbing

Fara Mutia
NIM. 2025573010034

M. REZA ZULMAN S.ST.,M.Sc
NIP. 19920501 2022031005

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah Praktikum Struktur Data dan Algoritma yang bertujuan untuk memberikan pemahaman tentang Linked List dengan menggunakan bahasa pemrograman JavaScript.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Bukit Rata, 31 Mei 2026
Penulis

Fara Mutia
NIM. 2025573010034

DAFTAR ISI

LEMBARAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
BAB II LANDASAN TEORI	2
2.1 Perbedaan Array dan Linked List	2
2.2 Jenis-jenis Linked List	2
BAB III METOLOGI PERCOBAAN	4
3.1 Mempersiapkan Project Praktikum	4
3.2 Mengerjakan Contoh, Latihan dan Tugas	4
3.3 Sumbit ke GitHub	15
BAB IV PEMBAHASAN DAN ANALISIS	17
4.1 PEMBAHASAN	17
4.2 ANALISA	18
BAB V PENUTUP.....	19
1.1 KESIMPULAN.....	19

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, pengelolaan data di dalam memori komputer memegang peranan yang sangat vital terhadap performa sebuah aplikasi. Sejauh ini, struktur data linier yang paling sering digunakan secara mendasar adalah Array. Array menyimpan elemen data secara bersebelahan (contiguous) di dalam memori, sehingga sangat unggul dalam melakukan akses data secara acak menggunakan indeks dengan kompleksitas waktu $O(1)$. Namun, Array memiliki kelemahan besar, yaitu ukurannya bersifat statis dan proses penyisipan (insert) maupun penghapusan (delete) data di tengah atau di awal memerlukan pergeseran (shifting) elemen yang memakan waktu $O(n)$.

Untuk mengatasi keterbatasan tersebut, konsep Linked List (Senarai Berantai) diperkenalkan sebagai alternatif struktur data linier yang dinamis. Berbeda dengan Array, Linked List menyimpan elemen datanya secara acak di memori di mana setiap elemen (disebut sebagai Node) saling terhubung menggunakan penunjuk atau pointer. Melalui mekanisme ini, Linked List sangat efisien dalam melakukan manipulasi data seperti penambahan dan penghapusan di awal list karena tidak memerlukan pergeseran memori sama sekali. Pemahaman mendalam mengenai bagaimana cara kerja Node, visualisasi alokasi memori, manipulasi pointer pada operasi dasar hingga lanjutan (seperti pembalikan arah list, deteksi siklus lintasan, serta implementasi struktur data turunan seperti Stack), menjadi landasan yang sangat krusial bagi mahasiswa dalam merancang algoritma yang efisien dan optimal.

1.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep Node sebagai unit penyusun Linked List.
2. Memahami perbedaan mendasar antara Array dan Linked List dari sisi memory.
3. Mengimplementasikan Singly Linked List lengkap menggunakan Class.
4. Mengimplementasikan operasi: prepend, append, insert, delete, search, traverse.
5. Mengimplementasikan reverse Linked List secara iterative.
6. Menganalisis Big O setiap operasi Linked List

BAB II LANDASAN TEORI

2.1 Perbedaan Array dan Linked List

Linked List adalah salah satu struktur data linier dinamis yang terdiri dari sejumlah simpul (Node) yang terhubung bersama secara berurutan. Setiap Node bertindak sebagai satu kesatuan unit penyusun yang secara umum membungkus dua komponen utama di dalamnya, yaitu: Data: Berisi nilai sesungguhnya yang ingin disimpan oleh pengguna. Next: Berisi referensi atau alamat memori (pointer) yang menunjuk ke objek Node berikutnya di dalam rantai senarai. Node pertama di dalam Linked List diidentifikasi sebagai Head, sementara Node terakhir dari list tersebut akan menunjuk ke nilai null sebagai penanda akhir dari struktur senarai (Singly Linked List). Beberapa perbedaan antara array dan linked list

1. Alokasi Memori: Array menggunakan alokasi memori statis dan bersebelahan (contiguous). Ukurannya harus dipesan sejak awal. Sebaliknya, Linked List menggunakan alokasi memori dinamis di mana elemen dapat diletakkan di mana saja di memori, dan ukurannya dapat berkembang atau menyusut secara fleksibel pada saat program berjalan.
2. Kompleksitas Waktu (Big O Notation): Akses Indeks: Array membutuhkan waktu $O(1)$ sedangkan Linked List membutuhkan waktu $O(n)$ karena harus menelusuri dari Head. Insert/Delete di Awal (Head): Array membutuhkan waktu $O(n)$ akibat efek shifting, sedangkan Linked List hanya membutuhkan $O(1)$ karena hanya perlu mengubah arah pointer head.
3. Insert/Delete di Tengah: Keduanya bernilai $O(n)$, namun Linked List unggul karena tidak memerlukan pergeseran fisik elemen di memori, hanya waktu pencarian lokasinya saja yang bernilai linear.

2.2 Jenis-jenis Linked List

Secara umum, Linked List dapat diimplementasikan ke dalam beberapa jenis struktur sesuai dengan arah penelusuran (traversal) dan arsitektur penyambungan antar-simpul yang dibutuhkan oleh sebuah sistem. Berikut adalah penjelasan mendalam mengenai jenis-jenis Linked List:

1. Singly Linked List (Senarai Berantai Tunggal)

Singly Linked List adalah variasi yang paling sederhana dan mendasar. Pada struktur ini, setiap node hanya memiliki satu buah variabel penunjuk (pointer) yang biasanya diberi nama next. Pointer ini menyimpan alamat memori dari node berikutnya dalam satu urutan linier. Penelusuran data pada Singly Linked List bersifat unidirectional atau hanya dapat bergerak maju satu arah, dimulai dari simpul pertama (Head) hingga berakhir di simpul terakhir yang menunjuk ke nilai null. Keunggulan utama dari jenis ini adalah penggunaan memori yang paling efisien karena setiap simpul hanya perlu mengalokasikan ruang untuk satu referensi alamat memori tambahan. Namun, kelemahannya terletak pada

keterbatasan penelusuran; program tidak dapat kembali ke node sebelumnya tanpa melakukan traversi ulang dari titik Head.

2. Doubly Linked List (Senarai Berantai Ganda)

Doubly Linked List mengatasi keterbatasan navigasi satu arah dengan menambahkan variabel penunjuk kedua pada setiap simpulnya, yaitu pointer prev (previous), di samping tetap mempertahankan pointer next. Dengan arsitektur ini, setiap simpul terhubung secara dua arah (bidirectional). Pointer next mengarah ke simpul sesudahnya, sedangkan pointer prev mengarah ke simpul sebelumnya. Selain itu, struktur ini biasanya dilengkapi dengan pointer Tail yang langsung mereferensikan simpul paling akhir. Kehadiran prev dan Tail memberikan fleksibilitas luar biasa, seperti kemampuan untuk melakukan pembacaan data secara terbalik dari belakang ke depan, serta mengoptimalkan operasi penambahan atau penghapusan di ujung akhir senarai menjadi sangat cepat dengan kompleksitas waktu konstan $O(1)$. Kompensasi dari fitur ini adalah penggunaan memori yang lebih besar, karena setiap simpul harus menyimpan dua buah referensi pointer sekaligus.

3. Circular Linked List (Senarai Berantai Melingkar)

Circular Linked List adalah variasi di mana simpul terakhir tidak menunjuk ke nilai null, melainkan mengarahkan kembali pointer next-nya ke simpul pertama (Head), sehingga membentuk sebuah struktur lintasan melingkar tanpa ujung akhir yang statis. Struktur melingkar ini dapat diterapkan baik pada Singly Linked List (menjadi Circular Singly Linked List) maupun pada Doubly Linked List (menjadi Circular Doubly Linked List di mana Tail.next menunjuk ke Head dan Head.prev menunjuk ke Tail). Struktur ini sangat berguna untuk aplikasi yang membutuhkan manajemen data berbasis perputaran giliran secara terus-menerus tanpa henti, seperti algoritma penjadwalan Round Robin pada sistem operasi, pemutar daftar lagu (playlist), atau mekanisme perputaran giliran pemain dalam sebuah game.

BAB III METOLOGI PERCOBAAN

3.1 Mempersiapkan Project Praktikum

1. Buka App Git Bash. Kemudian navigasikan ke root folder repository.

```
newuser@DESKTOP-JP3BR80 MINGW64 ~  
$ cd '/e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma'
```

2. Buat folder baru dengan nama praktikum-6 untuk menyimpan semua file pada pertemuan ini dan masuk kedalamnya.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanalgoritma (main)  
$ mkdir praktikum-6  
  
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanalgoritma (main)  
$ cd praktikum-6
```

3. Inisialisasi npm project.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanalgoritma/praktikum-6 (main)  
$ npm init -y  
Wrote to E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\prakti  
kum-6\package.json:  
  
{  
  "name": "praktikum-6",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

3.2 Mengerjakan Contoh, Latihan dan Tugas

3.2.1 Class Node & Linked List Dasar

1. Membuat file baru dengan nama 01-linked-list.js di dalam folder praktikum-6

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanalgoritma/praktikum-6 (main)  
$ touch 01-linked-list.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihannya.

```

1  class Node{
2      constructor(data){
3          this.data = data;
4          this.next = null;
5      }
6  }
7
8  class Linkedll{
9      constructor(){
10         this.head = null;
11         this.size = 0;
12     }
13
14     append(data){
15         const newNode = new Node(data);
16         if(!this.head){
17             this.head = newNode;
18         } else {
19             let current = this.head;
20             while (current.next) current = current.next;
21             current.next = newNode;
22         }
23         this.size++;
24     }
25
26     prepend(data){
27         const newNode = new Node(data)
28         newNode.next = this.head;
29         this.head = newNode;
30         this.size++;
31     }
32
33     insertAt(data, index){
34         if(index < 0 || index > this.size){
35             console.log(' Index di luar batas!');
36             return;
37         }
38         if (index === 0) { this.prepend(data); return; }
39         const newNode = new Node(data);
40         let current = this.head;
41         for (let i = 0; i < index - 1; i++) current = current.next;
42         newNode.next = current.next;
43         current.next = newNode;
44         this.size++;
45     }
46
47     delete(data){
48         if(!this.head) return false;
49         if(this.head.data === data){
50             this.head = this.head.next;
51             this.size--;
52             return true;
53         }
54         let current = this.head;
55         while(current.next){
56             if (current.next){
57                 current.next = current.next.next;
58                 this.size--;
59                 return true;
60             }
61             current = current.next;
62         }
63         return false;
64     }
65
66     search(data){
67         let current = this.head;
68         let index = 0;
69         while(current){
70             if (current.data = data) return index;
71             current = current.next;
72             index++;
73         }
74         return -1;
75     }
76
77     print(){
78         if (!this.head){console.log(' [ll kosong]'); return; }
79         let result = '';
80         let current = this.head;
81         while (current) {
82             result += current.next ? ` [${current.data}] + ` : ` [${current.data}]`;
83             current = current.next;
84         }
85         console.log(' ', result, ' (size: ${this.size})');
86     }
87
88     reverse(){
89         let prev = null;
90         let current = this.head;
91         while(current){
92             const next = current.next;
93             current.next = prev;
94             prev = current;
95             current = next;
96         }
97         this.head = prev;
98     }
99 }

```

```

96     toArray() {
97         const arr = [];
98         let current = this.head;
99         while (current) { arr.push(current.data); current = current.next; }
100         return arr;
101     }
102     //latihannnn
103     getAt(index) {
104         if (index < 0 || index >= this.size) {
105             return null;
106         }
107         let current = this.head;
108         for (let i = 0; i < index; i++) {
109             current = current.next;
110         }
111         return current.data;
112     }
113
114     deleteAt(index) {
115         if (index < 0 || index >= this.size) {
116             console.log(' Index di luar batas untuk menghapus!');
117             return null;
118         }
119
120         let deletedNodeData;
121
122         if (index === 0) {
123             deletedNodeData = this.head.data;
124             this.head = this.head.next;
125         } else {
126             let current = this.head;
127             for (let i = 0; i < index - 1; i++) {
128                 current = current.next;
129             }
130             deletedNodeData = current.next.data;
131             current.next = current.next.next;
132         }
133
134         this.size--;
135         return deletedNodeData;
136     }
137
138     indexOf(data) {
139         return this.search(data);
140     }
141
142     isEmpty() {
143         return this.size === 0;
144     }

```

```

147     clear() {
148         this.head = null;
149         this.size = 0;
150     }
151 }
152
153
154 const ll = new Linkedll();
155
156 console.log('=== Append ===');
157 ll.append(10); ll.append(20);
158 ll.append(30); ll.append(40);
159 ll.print();
160
161 console.log('\n=== Prepend ===');
162 ll.prepend(5);
163 ll.print();
164
165 console.log('\n=== Insert di indeks 2 ===');
166 ll.insertAt(15, 2);
167 ll.print();
168
169 console.log('\n=== Search ===');
170 console.log(' Indeks nilai 20:', ll.search(20));
171 console.log(' Indeks nilai 99:', ll.search(99));
172
173 console.log('\n=== Delete 20 ===');
174 ll.delete(20);
175 ll.print();
176
177 console.log('\n=== Reverse ===');
178 ll.reverse();
179 ll.print();
180
181 console.log('=== Uji isEmpty() awal ===');
182 console.log('Apakah list kosong?', ll.isEmpty());
183

```

```

185 console.log('\n=== Memasukkan Data Baru ===');
186 ll.append('Apel');
187 ll.append('Mangga');
188 ll.append('Pisang');
189 ll.append('Jeruk');
190 ll.print();
191 console.log('Apakah list kosong sekarang?', ll.isEmpty());
192
193 console.log('\n=== Uji getAt(index) ===');
194 console.log('Data di indeks 0:', ll.getAt(0));
195 console.log('Data di indeks 2:', ll.getAt(2));
196 console.log('Data di indeks 99 (invalid):', ll.getAt(99));
197
198 console.log('\n=== Uji indexOf(data) ===');
199 console.log('Indeks dari "Mangga":', ll.indexOf('Mangga'));
200 console.log('Indeks dari "Semangka":', ll.indexOf('Semangka'));
201
202 console.log('\n=== Uji deleteAt(index) ===');
203 console.log('Hapus data di indeks 2 ("Pisang")...');
204 ll.deleteAt(2);
205 ll.print();
206
207 console.log('Hapus data di indeks 0 ("Apel")...');
208 ll.deleteAt(0);
209 ll.print();
210
211 console.log('\n=== Uji clear() ===');
212 console.log('Mengosongkan semua node...');
213 ll.clear();
214 ll.print();
215 console.log('Apakah list kosong setelah clear()?', ll.isEmpty());

```

2.2.2 Deteksi Siklus dan Merge

1. Membuat file baru dengan nama 02-ll-lanjutan.js di dalam folder praktikum-6

```

newuser@DESKTOP-JP3BRB0 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadana/algorithm/praktikum-6 (main)
$ touch 02-ll-lanjutan.js

```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```

1  class Node {
2      constructor(d){
3          this.data = d;
4          this.next = null;
5      }
6  }
7  function adaSiklus(head){
8      let lambat = head;
9      let cepat = head;
10     while (cepat && cepat.next){
11         lambat = lambat.next;
12         cepat = cepat.next.next;
13         if (lambat === cepat) return true;
14     }
15     return false;
16 }
17 function mergeTerurut(head1, head2){
18     const dummy = new Node(0);
19     let current = dummy;
20     while (head1 && head2) {
21         if (head1.data <= head2.data) {
22             current.next = head1;
23             head1 = head1.next;
24         } else {
25             current.next = head2;
26             head2 = head2.next;
27         }
28         current = current.next;
29     }
30     current.next = head1 || head2;
31     return dummy.next;
32 }
33 function buatList(arr){
34     if (!arr.length) return null;
35     const head = new Node(arr[0]);
36     let cur = head;
37     for (let i = 1; i < arr.length; i++){
38         cur.next = new Node(arr[i]);
39         cur = cur.next;
40     }
41     return head;
42 }
43
44 function cetakList(head){
45     let s='', cur = head;
46     while(cur){
47         s += cur.next?`${cur.data}->` : `${cur.data}`; cur = cur.next;}
48     console.log(' ', s);
49 }
50
51 const A = buatList([1,2,3,4,5]);
52 console.log('Ada siklus (seharusnya false) :', adaSiklus(A));
53
54 A.next.next.next.next.next = A.next;
55 console.log('Ada siklus (seharusnya true) :', adaSiklus(A));
56
57 const L1 = buatList([1,3,5,7,9]);
58 const L2 = buatList([2,4,6,8,10]);
59 console.log('\nList 1:'); cetakList(L1);
60 console.log('List 2:'); cetakList(L2);
61 const merged = mergeTerurut(L1, L2);
62 console.log('Merged:'); cetakList(merged);

```

2.2.3 Implementasi Stack dengan Linked List

1. Buat file baru dengan nama 03-stack-ll.js di dalam folder praktikum-6.

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-6 (main)
$ touch 03-stack-ll.js

```

2. Lalu buka file tersebut di VS Code. Kerjakan latihannya.

```
1  class Node{
2      constructor(data){
3          this.data = data;
4          this.next = null;
5      }
6  }
7  class Linkedll{
8      constructor(){
9          this.head = null;
10         this.length = 0;
11     }
12     prepend(data){
13         const node = new Node(data);
14         node.next = this.head;
15         this.head = node;
16         this.length++;
17     }
18     removeHead(){
19         if (!this.head) return null;
20         const removed = this.head.next;
21         this.length--;
22         return removed;
23     }
24     peekHead(){
25         return this.head? this.head.data : null;
26     }
27     getSize(){
28         return this.length;
29     }
30     isEmpty(){
31         return this.length === 0;
32     }
33     toArray(){
34         const result = [];
35         let current = this.head;
36         while(current){
37             result.push(current.data);
38             current = current.next;
39         }
40         return this;
41     }
42 }
```

```
44 class Stack{
45     constructor(){
46         this.list = new Linkedll();
47     }
48 }
49 push(data){
50     this.list.prepend(data);
51     console.log(` push(${JSON.stringify(data)}) + stack: [${this.list.toArray()}]`);
52 }
53 pop() {
54     if (this.isEmpty()) {
55         console.log(" pop() → Stack kosong! Tidak ada yang bisa di-pop.");
56         return null;
57     }
58     const removed = this._list.removeHead();
59     console.log(` pop() → ${JSON.stringify(removed)} | stack: [${this.list.toArray()}]`);
60     return removed;
61 }
62 peek() {
63     const top = this.list.peekHead();
64     console.log(` peek() → ${top !== null ? JSON.stringify(top) : "null (kosong)"} `);
65     return top;
66 }
67 isEmpty() {
68     return this.list.isEmpty();
69 }
70 size() {
71     return this.list.getSize();
72 }
73 print() {
74     const arr = this.list.toArray();
75     if (arr.length === 0) {
76         console.log(" Stack: (kosong)");
77     } else {
78         console.log(` Stack (top → bottom): [${arr.join(", ")}]`);
79     }
80 }
81 }
82 }
```

```

83 console.log("=".repeat(60));
84 console.log("  SIMULASI UNDO/REDO DENGAN STACK (LinkedList)");
85 console.log("=".repeat(60));
86 const aksi = [
87   "Fara ",
88   "belajar ",
89   "Linked List "
90 ];
91 const undoStack = new Stack();
92 const redoStack = new Stack();
93 console.log("\nMelakukan aksi:");
94 for (const a of aksi) {
95   undoStack.push(a);
96 }
97 console.log("\nStatus setelah semua aksi:");
98 undoStack.print();
99 console.log(`  Ukuran stack: ${undoStack.size()}`);
100
101 // --- Undo beberapa kali ---
102 console.log("\nUndo 3 kali:");
103 for (let i = 0; i < 3; i++) {
104   const undone = undoStack.pop();
105   if (undone) {
106     redoStack.push(undone); // simpan ke redo
107   }
108 }
109 console.log("\nUndo Stack setelah 3x undo:");
110 undoStack.print();
111
112 console.log("\nRedo Stack (aksi yang di-undo):");
113 redoStack.print();
114 // --- Redo sekali ---
115 console.log("\nRedo 1 kali:");
116 const redone = redoStack.pop();
117 if (redone) {
118   undoStack.push(redone);
119 }
120 console.log("\nUndo Stack setelah 1x redo:");
121 undoStack.print();
122 console.log("\nRingkasan:");
123 console.log(`  isEmpty (undoStack): ${undoStack.isEmpty()}`);
124 console.log(`  size      (undoStack): ${undoStack.size()}`);
125 console.log(`  peek      (undoStack):`);
126 undoStack.peek();

```

2.2.4 Tugas Praktikum

1. Buat folder tugas/

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-6 (main)
$ mkdir tugas

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-6 (main)
$ cd tugas

```

2. Lalu tambahkan dua file tugas di dalam folder tugas/ tersebut yaitu tugas-1.js dan tugas-2.js.

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-6/tugas (main)
$ touch tugas-1.js

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-6/tugas (main)
$ touch tugas-2.js

```

3. Program tugas 1.

```

1  class Node {
2      constructor(data) {
3          this.data = data;
4          this.next = null;
5          this.prev = null;
6      }
7  }
8
9  class DoublyLinkedList {
10     constructor() {
11         this.head = null;
12         this.tail = null;
13         this.length = 0;
14     }
15     // -----
16     // append(data) – tambah di akhir
17     // Big O: O(1) karena ada pointer tail
18     // Bukti: kita langsung akses this.tail tanpa traversal
19     append(data) {
20         const node = new Node(data);
21         if (!this.head) {
22             this.head = node;
23             this.tail = node;
24         } else {
25             node.prev = this.tail;
26             this.tail.next = node;
27             this.tail = node;
28         }
29         this.length++;
30         return this;
31     }
32     // -----
33     // prepend(data) – tambah di awal
34     // Big O: O(1)
35     prepend(data) {
36         const node = new Node(data);
37         if (!this.head) {
38             this.head = node;
39             this.tail = node;
40         } else {
41             node.next = this.head;
42             this.head.prev = node;
43             this.head = node;
44         }
45         this.length++;
46         return this;
47     }

```

```

48 // -----
49 // insertAt(index, data) - sisip pada posisi tertentu
50 // Big O: O(n) - perlu traversal ke index
51 insertAt(index, data) {
52   if (index < 0 || index > this.length) {
53     console.log(`insertAt: index ${index} di luar batas (0-${this.length})`);
54     return this;
55   }
56   if (index === 0) return this.prepend(data);
57   if (index === this.length) return this.append(data);
58
59   const node = new Node(data);
60   let current = this.head;
61   for (let i = 0; i < index; i++) {
62     current = current.next;
63   }
64   const prev = current.prev;
65   node.next = current;
66   node.prev = prev;
67   prev.next = node;
68   current.prev = node;
69   this.length++;
70   return this;
71 }

```

```

72 // -----
73 // delete(data) - hapus node pertama yang nilainya = data
74 // Big O: O(n) - perlu traversal untuk mencari
75 delete(data) {
76   if (!this.head) return null;
77   let current = this.head;
78   while (current) {
79     if (current.data === data) {
80       if (current === this.head && current === this.tail) {
81         this.head = null;
82         this.tail = null;
83       } else if (current === this.head) {
84         this.head = current.next;
85         this.head.prev = null;
86       } else if (current === this.tail) {
87         this.tail = current.prev;
88         this.tail.next = null;
89       } else {
90         current.prev.next = current.next;
91         current.next.prev = current.prev;
92       }
93       this.length--;
94       return current.data;
95     }
96     current = current.next;
97   }
98   return null;
99 }
100 // -----
101 // reverse() - balik urutan linked list in-place
102 // Big O: O(n)
103 reverse() {
104   if (!this.head || this.length === 1) return this;
105
106   let current = this.head;
107   while (current) {
108     const temp = current.next;
109     current.next = current.prev;
110     current.prev = temp;
111     current = temp;
112   }
113   const temp = this.head;
114   this.head = this.tail;
115   this.tail = temp;
116   return this;
117 }

```

4. Program tugas 2.

```
1  class Node {
2    constructor(data) {
3      this.data = data;
4      this.next = null;
5    }
6  }
7
8  function buatList(...nilai) {
9    if (nilai.length === 0) return null;
10   const head = new Node(nilai[0]);
11   let current = head;
12   for (let i = 1; i < nilai.length; i++) {
13     current.next = new Node(nilai[i]);
14     current = current.next;
15   }
16   return head;
17 }
18
19 function cetakList(head) {
20   const items = [];
21   let cur = head;
22   while (cur) {
23     items.push(cur.data);
24     cur = cur.next;
25   }
26   return items.length ? "[" + items.join(",") + "]" : "(kosong)";
27 }
28
29 function palindromLL(head) {
30   const arr = [];
31   let cur = head;
32   while (cur) {
33     arr.push(cur.data);
34     cur = cur.next;
35   }
36   let kiri = 0;
37   let kanan = arr.length - 1;
38   while (kiri < kanan) {
39     if (arr[kiri] !== arr[kanan]) return false;
40     kiri++;
41     kanan--;
42   }
43   return true;
44 }
45
```

```

46 function hapusNDariAkhir(head, n) {
47     const dummy = new Node(0);
48     dummy.next = head;
49
50     let fast = dummy;
51     let slow = dummy;
52
53     for (let i = 0; i <= n; i++) {
54         if (!fast) return head;
55         fast = fast.next;
56     }
57
58     while (fast) {
59         fast = fast.next;
60         slow = slow.next;
61     }
62
63     slow.next = slow.next.next;
64
65     return dummy.next;
66 }
67
68 function tengahLinkedList(head) {
69     if (!head) return null;
70
71     let slow = head;
72     let fast = head;
73
74     while (fast && fast.next) {
75         slow = slow.next;
76         fast = fast.next.next;
77     }
78
79     return slow;
80 }
81
82 console.log("\npalindromLL(head)");
83 console.log("-".repeat(40));
84
85 const kasusP = [
86     { list: buatList(1, 2, 3, 2, 1), label: "[1+2+3+2+1]" },
87     { list: buatList(1, 2, 2, 1), label: "[1+2+2+1]" },
88     { list: buatList(1, 2, 3, 4, 5), label: "[1+2+3+4+5]" },
89     { list: buatList(7), label: "[7]" },
90     { list: buatList(1, 2), label: "[1+2]" },
91 ];

```

```

93 kasusP.forEach(({ list, label, expect }) => {
94   const hasil = palindromLL(list);
95   console.log(`palindromLL(${label}) -> ${hasil}`);
96 });
97
98 console.log("\nhapusNDariAkhir(head, n)");
99 console.log("-".repeat(40));
100
101 const kasusH = [
102   { list: buatList(1, 2, 3, 4, 5), n: 2, label: "[1+2+3+4+5]",
103   { list: buatList(1, 2, 3, 4, 5), n: 1, label: "[1+2+3+4+5]",
104   { list: buatList(1, 2, 3, 4, 5), n: 5, label: "[1+2+3+4+5]",
105   { list: buatList(1), n: 1, label: "[1]",
106   { list: buatList(1, 2), n: 2, label: "[1+2]",
107 ];
108
109 kasusH.forEach(({ list, n, label, expect }) => {
110   const hasil = hapusNDariAkhir(list, n);
111   const strHasil = cetakList(hasil);
112   console.log(`hapusNDariAkhir(${label}, ${n}) -> ${strHasil}`);
113 });
114
115 console.log("\ntengahLinkedList(head)");
116 console.log("-".repeat(40));
117
118 const kasusT = [
119   { list: buatList(1, 2, 3, 4, 5), label: "[1+2+3+4+5]", expect: 3, note: "ganjil + tengah" },
120   { list: buatList(1, 2, 3, 4), label: "[1+2+3+4]", expect: 3, note: "genap + tengah ke-2" },
121   { list: buatList(1, 2, 3), label: "[1+2+3]", expect: 2, note: "ganjil" },
122   { list: buatList(1, 2), label: "[1+2]", expect: 2, note: "genap + tengah ke-2" },
123   { list: buatList(42), label: "[42]", expect: 42, note: "satu node" },
124 ];
125
126 kasusT.forEach(({ list, label, expect, note }) => {
127   const hasil = tengahLinkedList(list);
128   const nilai = hasil ? hasil.data : null;
129   console.log(`tengahLinkedList(${label}) -> node(${nilai}) (expect: ${expect}) - ${note}`);
130 });

```

3.3 Sumbit ke GitHub

Setelah semua tugas, latihan, dan contoh selesai, lakukan push ke *repository* GitHub.

1. Ketik `cd ..` di Git Bush agar tidak berada dalam `/praktikum-6` tetapi di root folder repository. Karena Git diinisialisasi di folder utama (root folder)

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-6 (main)
$ cd ..

```

2. Ketik `git status` untuk cek status file

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   praktikum-2/01-variabel.js
    modified:   praktikum-2/02-tipe-data.js
    modified:   praktikum-2/03-operator.js
    modified:   praktikum-2/04-kondisional.js
    modified:   praktikum-2/05-perulangan.js
    modified:   praktikum-2/tugas/tugas1.js
    modified:   praktikum-2/tugas/tugas2.js
    modified:   praktikum-3/tugas/tugas-2.js
    modified:   praktikum-4/02-destructuring-spread.js
    modified:   praktikum-4/03-class-dasar.js
    modified:   praktikum-4/04-inheritance.js
    modified:   praktikum-4/tugas/tugas-1.js
    modified:   praktikum-4/tugas/tugas-2.js
    modified:   praktikum-5/02-kelas-big-o.js
    modified:   praktikum-5/03-space-complexity.js
    modified:   praktikum-5/latihan-1.js
    modified:   praktikum-5/tugas/tugas-1.js
    modified:   praktikum-5/tugas/tugas-2.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    praktikum-6/

no changes added to commit (use "git add" and/or "git commit -a")

```

3. Lalu ketik `git add praktikum -6/` untuk menambahkan

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ git add praktikum-6/
warning: in the working copy of 'praktikum-6/package.json', LF will be replaced
by CRLF the next time Git touches it
```

4. Buat commit dengan pesan yang deskriptif. Seperti `git commit -m "Praktikum 6: Linked List"`.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ git commit -m "Praktikum 6: Linked List"
[main 78f87b9] Praktikum 6: Linked List
6 files changed, 773 insertions(+)
create mode 100644 praktikum-6/01-linked-list.js
create mode 100644 praktikum-6/02-ll-lanjutan.js
create mode 100644 praktikum-6/03-stack-ll.js
create mode 100644 praktikum-6/package.json
create mode 100644 praktikum-6/tugas/tugas-1.js
create mode 100644 praktikum-6/tugas/tugas-2.js
```

5. Lalu lakukan push ke repository GitHub.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 5.62 KiB | 338.00 KiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/faraa12/202573010034_strukturdatadanalgoritma.git
33fc419..78f87b9 main -> main
```

BAB IV PEMBAHASAN DAN ANALISIS

4.1 PEMBAHASAN

Praktikum dibagi menjadi tiga file latihan utama dan dua program tugas mandiri:

1. File 01-linked-list.js (Class Node & Linked List Dasar):

Berhasil dibuat struktur class Node dan class LinkedList. Logika manipulasi pointer diterapkan pada operasi append (menambahkan di akhir melalui traversi hingga `current.next` bernilai null), prepend (memasukkan data baru di posisi head dengan Big O $O(1)$), insertAt (menyisipkan simpul pada indeks tertentu), dan delete (memutus rantai simpul dengan melompati objek yang dicari via instruksi `current.next = current.next.next`). Fitur tambahan dari Latihan 1 juga sukses diintegrasikan, meliputi pembuatan fungsi pencarian posisi data (`getAt`, `indexOf`), pembersihan list (`clear`), penanda kekosongan (`isEmpty`), dan fungsi penghapusan berbasis indeks (`deleteAt`). Selain itu, operasi pembalikan urutan senarai secara iteratif menggunakan tiga bantuan variabel penunjuk (`prev`, `current`, `next`) pada fungsi `reverse()` berhasil mengarahkan kembali seluruh rantai tanpa menduplikasi data di memori.

2. File 02-ll-lanjutan.js (Deteksi Siklus dan Merge):

Implementasi fungsi `adaSiklus(head)` menggunakan Floyd's Algorithm terbukti bekerja valid. Ketika pengujian dilakukan pada list linier normal A, fungsi mengembalikan nilai false. Namun, setelah dilakukan manipulasi rantai buatan di mana node terakhir sengaja disambungkan kembali ke node kedua (`A.next.next.next.next.next = A.next`), algoritma berhasil mendeteksi perputaran tersebut dan mengembalikan nilai true. Untuk fungsi `mergeTerurut(head1, head2)`, penggunaan teknik dummy node membantu proses penyatuan dua buah objek Linked List yang telah terurut secara ascending (List 1 dan List 2) menjadi satu kesatuan senarai baru yang rapi.

3. File 03-stack-ll.js (Implementasi Stack dengan Linked List):

Struktur data Stack sukses dibuat dengan menggunakan Singly Linked List sebagai media penyimpanan internalnya (Composition). Agar memenuhi karakteristik utama Stack, yaitu Last In First Out (LIFO), seluruh operasi manipulasi data dipaksa berjalan dalam performa konstan $O(1)$. Operasi `push(data)` memanfaatkan fungsi `prepend()` (memasukkan di posisi terdepan/top), sementara operasi `pop()` memanfaatkan fungsi `removeHead()` (menghapus elemen terdepan/top). Program ini diuji secara interaktif melalui simulasi mekanisme Undo/Redo pada deretan riwayat aksi pengetikan mahasiswa.

4. File tugas/tugas-1.js (Doubly Linked List):

Berhasil diimplementasikan simpul dua arah yang memiliki referensi objek data ke depan (`this.next`) dan ke belakang (`this.prev`). Kehadiran property pointer `this.tail` pada deklarasi awal constructor kelas berhasil mengoptimalkan fungsi `append()` dari yang semula bernilai linear $O(n)$ pada Singly Linked List menjadi bernilai konstan

$O(1)$, sebab kode program dapat langsung mengakses ujung akhir senarai secara instan tanpa perlu melakukan proses perulangan (looping traversal) dari awal. File tugas/tugas-2.js (Soal Klasik Linked List): Fungsi `palindromLL(head)` berhasil mengidentifikasi kesamaan pola bacaan depan-belakang dengan menyalin isi list ke bentuk array penampung terlebih dahulu. Fungsi `hapusDariAkhir(head, n)` berhasil memutus simpul ke- n dari belakang memanfaatkan prinsip jarak rentang konstan antardua pointer (slow dan fast pointer). Fungsi `tengahLinkedList(head)` secara cerdas mampu mengembalikan titik simpul tengah senarai secara presisi menggunakan kecepatan langkah ganda dari penunjuk fast.

4.2 ANALISA

Berdasarkan hasil eksekusi program, efisiensi waktu (Big O) dari setiap operasi sangat dipengaruhi oleh arsitektur penunjuk (pointer) yang digunakan. Operasi `prepend` (menambahkan di awal) pada Singly maupun Doubly Linked List terbukti sangat efisien dengan kompleksitas waktu konstan $O(1)$ karena hanya mengubah referensi pointer Head tanpa dipengaruhi oleh panjang data. Efisiensi konstan $O(1)$ ini juga berhasil diterapkan pada metode `push` dan `pop` di struktur data Stack karena manipulasi data sengaja dipusatkan di posisi Head untuk memenuhi sifat Last In First Out (LIFO).

Sebaliknya, perbedaan signifikan terjadi pada operasi `append` (menambahkan di akhir). Pada Singly Linked List, `append` memerlukan waktu linear $O(n)$ karena program harus melakukan perulangan (traversal) dari awal untuk menemukan ujung list. Namun, pada Doubly Linked List, kehadiran pointer Tail berhasil memangkas waktu operasi `append` menjadi konstan $O(1)$ karena program dapat langsung mengakses ujung akhir senarai secara instan.

Sementara itu, operasi pencarian dan manipulasi di posisi acak seperti `insertAt`, `delete`, dan `search` pada kedua jenis Linked List tetap bernilai linear $O(n)$ karena tidak mendukung akses acak berbasis indeks seperti Array, sehingga memaksa program memeriksa simpul satu per satu. Untuk algoritma lanjutan, fungsi `reverse` memerlukan waktu $O(n)$ untuk membalikkan arah seluruh pointer, sedangkan Floyd's Cycle Detection Algorithm teranalisis sangat optimal dengan waktu $O(n)$ dan kompleksitas ruang konstan $O(1)$ karena hanya memanfaatkan dua pointer bantu tanpa membebani memori baru.

BAB V PENUTUP

1.1 KESIMPULAN

Praktikum ini berhasil membuktikan bahwa Linked List merupakan struktur data linier dinamis yang sangat fleksibel dan efisien dalam manajemen memori karena alokasinya dilakukan secara acak pada saat program berjalan (runtime), berbeda dengan Array yang bersifat statis dan kaku. Melalui implementasi menggunakan JavaScript (Node.js), dapat disimpulkan bahwa operasi manipulasi di awal senarai (prepend) memiliki efisiensi performa konstan $O(1)$, menjadikannya fondasi ideal untuk struktur data Stack yang berbasis LIFO. Selain itu, optimalisasi struktur melalui penambahan pointer Tail pada Doubly Linked List terbukti mampu memangkas kompleksitas waktu operasi di akhir senarai (append) dari linear $O(n)$ menjadi konstan $O(1)$. Terakhir, penerapan Floyd's Cycle Detection Algorithm (Fast & Slow Pointer) teruji valid sebagai solusi terbaik untuk mendeteksi anomali lintasan melingkar pada senarai berantai secara cepat dengan penggunaan ruang memori yang sangat minimal.